Simple Constructs Big Power
Master these fundamentals!

Java Language Refresher – Chapter 2

Variables • Primitive Types • Operators • Conditions • Loops

Key Building Blocks for Spring Boot and Advanced Java

1 Variables & Scope

Store values and control visibility



- Declaration, initialization, assignment
- Meaningful names (lowerCamelCase)
- final and var keywords
- Scope: local, block, method
- Variables must be assigned before use

```
int score = 10;   // declaration + init
int bonus = 3;
score = score + bonus;  // 13
final double PI = 3.14;
var name = "Java";
```

Scope Examples

- Method scope (visible inside method)
- Block scope (visible inside { } only)
- Cannot use variable outside its scope

Common Mistakes

- Using variable before initialization
- Accessing variable outside its scope

2 Primitive Types

8 built-in types with fixed ranges

Type	Size	Range (Approximate)
byte	8 bits	-128 to 127
short	16 bits	-32,768 to 32,767
int	32 bits	-2.1 billion to 2.1 billion
long	64 bits	-9.2e18 to 9.2e18
float	32 bits	~6-7 decimal digits
double	64 bits	~15-16 decimal digits
char	16 bits	0 to 65,535 (Unicode)
boolean	varies	true or false

3 Operators

Perform operations on values

Category	Operators	Example	Result
Arithmetic	+ - * / %	7 % 3	1
Assignment	= += -= *= /=	x *= 5	x = x + 5
Comparison	== != > < >= <=	5 == 5	true
Logical	&& | !	true && false	false
Bitwise	& | ^ ~ << >>	5 << 1	10
Unary	+ - ++ -- !	++x	x = x + 1
Ternary	? :	age > 18 ? "Adult" : "Minor"	

Quick Examples

```
int a = 10, b = 3;
System.out.println(a + b);   // 13
System.out.println(a % b);   // 1
System.out.println(a > b);   // true
```

Operator Precedence (High → Low)

() → unary → * / % → + - → << >> → < <= > >= → == != → & → ^ → | → && → || → ? : → = += -= ...

4 Type Casting & Conversion

Change a value from one type to another

Automatic Widening (safe)

double
float
long
int
short
byte

Explicit Narrowing (cast needed)

Examples

```
int i = 100;
long l = i;          // widening (auto)
double d = i;        // widening (auto)

int n = (int) 9.7;   // narrowing (cast)
byte b = (byte) 130; // 130 → -126 (overflow)
```

Important Pionts

- Widening is automatic and safe.
- Narrowing needs an explicit cast.
- int arithmetic promotes to int.
- Be careful with overflow when narrowing.

5 Strings

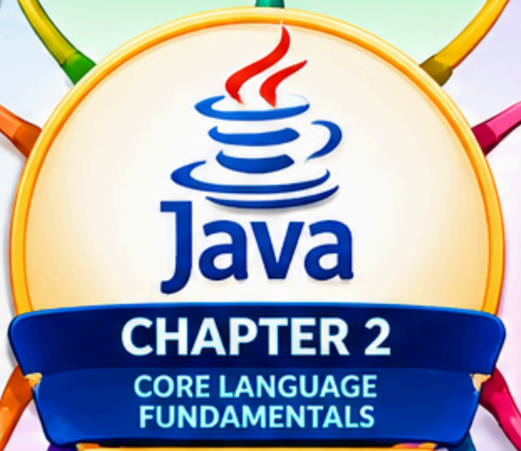
Work with text (not a primitive)

```
String s = "Java";
int len = s.length();              // 4
char ch = s.charAt(1);             // 'a'
String upper = s.toUpperCase();    // "JAVA"
String sub = s.substring(1, 3);    // "av"
boolean eq = s.equals("Java");     // true
```

Common String Operations

- length() – number of characters
- charAt(i) – character at index
- substring(start, end) – part of string
- equals() – compare content
- + – concatenation
- toUpperCase(), toLowerCase()

String is immutable
Methods create new strings. Original string does not change.



6 Conditions & Switch

Make decisions in your code

if – else

```
int age = 18;
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

Ternary Operator

```
String type = (age >= 18) ?
              "Adult" : "Minor";
```

switch (Java 17)

```
switch (day) {
    case 1 -> System.out.println("Mon");
    case 2,3,4,5 -> System.out.println("Weekday");
    default -> System.out.println("Weekend");
}
```

When to use?

- if – simple conditions
- else if – multiple conditions
- switch – many fixed values
- ternary – short expression

Common Mistakes

- Missing break in switch (pre Java 14)
- Using = instead of ==
- Incorrect condition logic

7 Loops

Repeat a block of code

for loop

```
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}
```

Output:

```
1
2
3
```

while loop

```
int i = 1;
while (i <= 3) {
    System.out.println(i);
    i++;
}
```

Output:

```
1
2
3
```

do-while loop

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 3);
```

Output:

```
1
2
3
```

Loop Controls

break	- exit loop
continue	- skip to next iteration
nested loops	- loop inside loop

Common Mistakes

- Infinite loop (condition never false)
- Off-by-one errors
- Forgetting to update counter
- Misusing break/continue

8 Quick Examples & Output

See it all together

```
int age = 46;
double price = 19.95;
boolean isMember = true;
char grade = 'A';
System.out.println(age + " | " + price + " | " + isMember + " | " + grade);
System.out.println("Next age: " + (age + 1));
```

Output:

```
46 | 19.95 | true | A
Next age: 47
```

Key Takeaways

- Choose the right type for the right value.
- Understand operator precedence.
- Use meaningful variable names.
- Write clear and readable conditions and loops.
- Avoid common mistakes.